

Course: Fundamentals of Generative AI and Large Language Models: Theory and Practice

Week 1: Fundamentals of Deep Learning for Generative AI

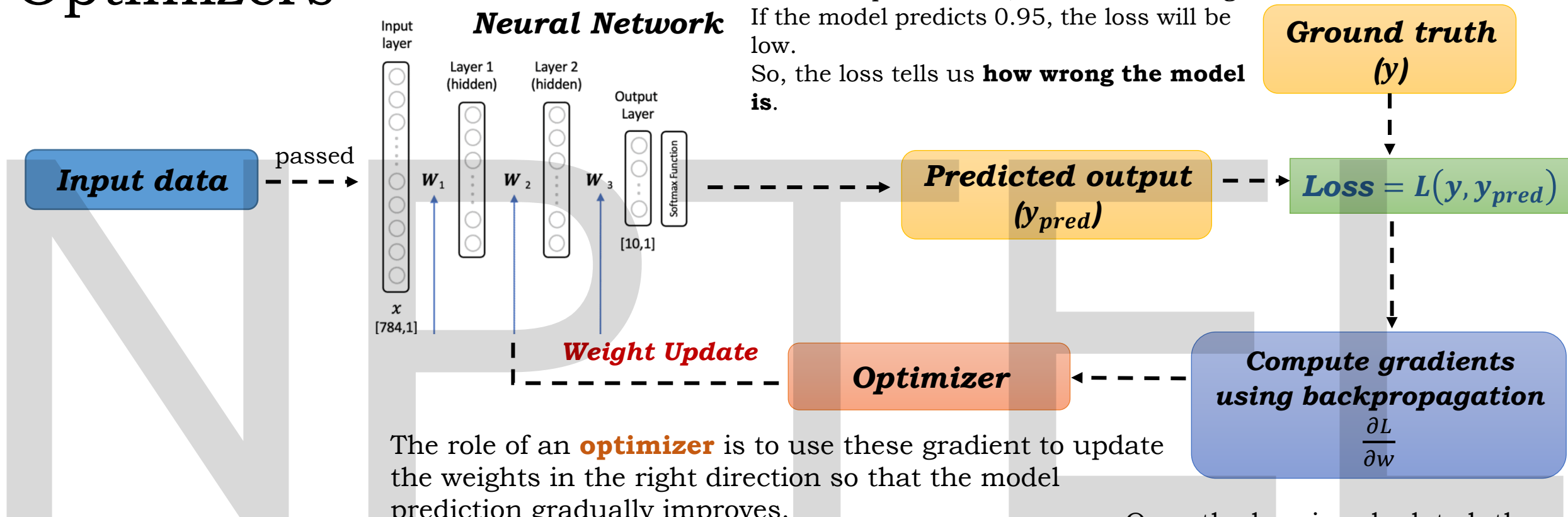
Today's Session Overview

Deep Learning Foundations for Generative AI

- Optimizers for training deep neural networks

NPTREL

Optimizers



For example, if the actual answer is 1 and the model predicts 0.2, the loss will be high. If the model predicts 0.95, the loss will be low. So, the loss tells us **how wrong the model is**.

The role of an **optimizer** is to use these gradient to update the weights in the right direction so that the model prediction gradually improves.

The main objective of optimization is:

Minimize the loss function

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

 $\frac{\partial L}{\partial W}$, the value can be >0 or <0 either positive or negative.

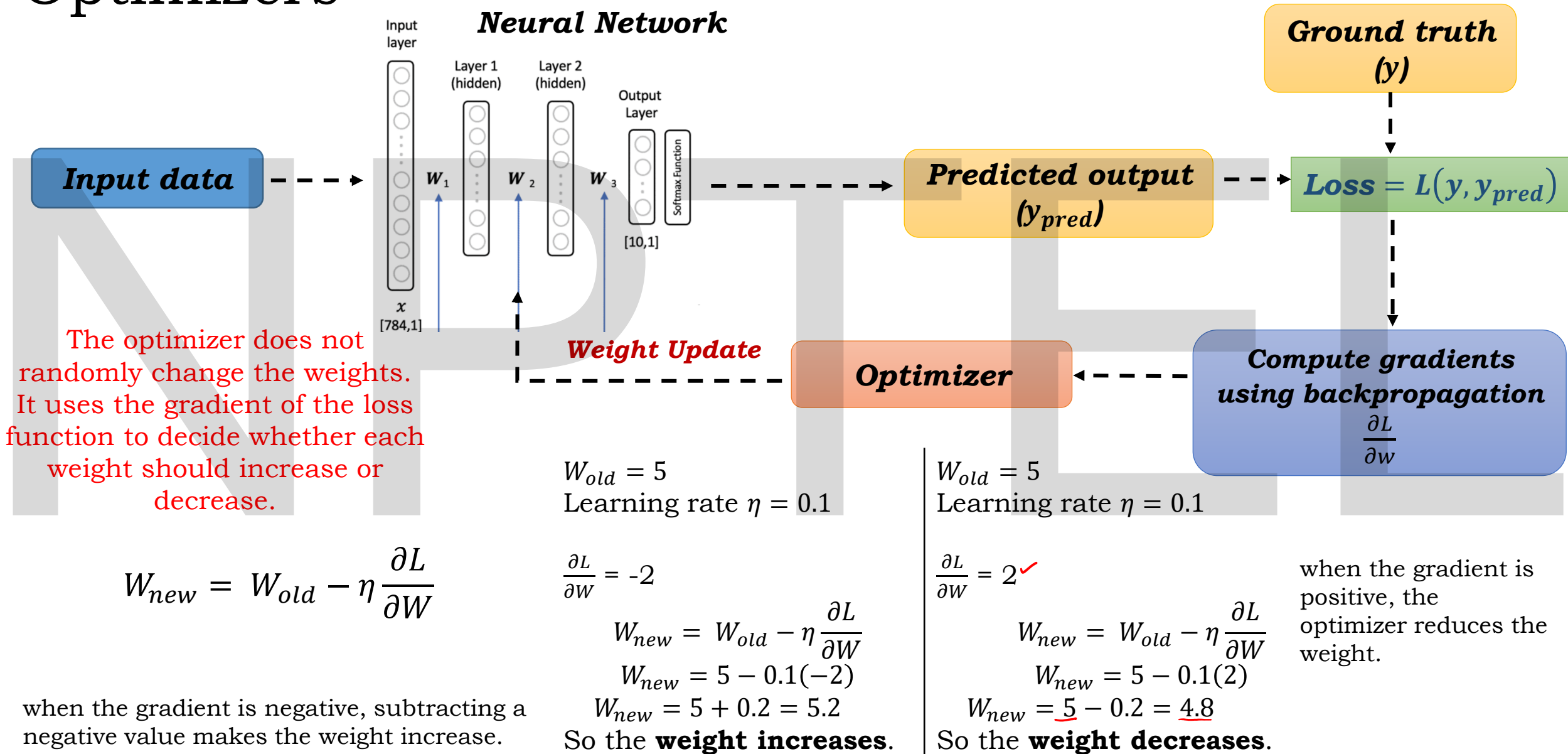
We subtract the gradient because we want to move in the direction where the loss decreases.

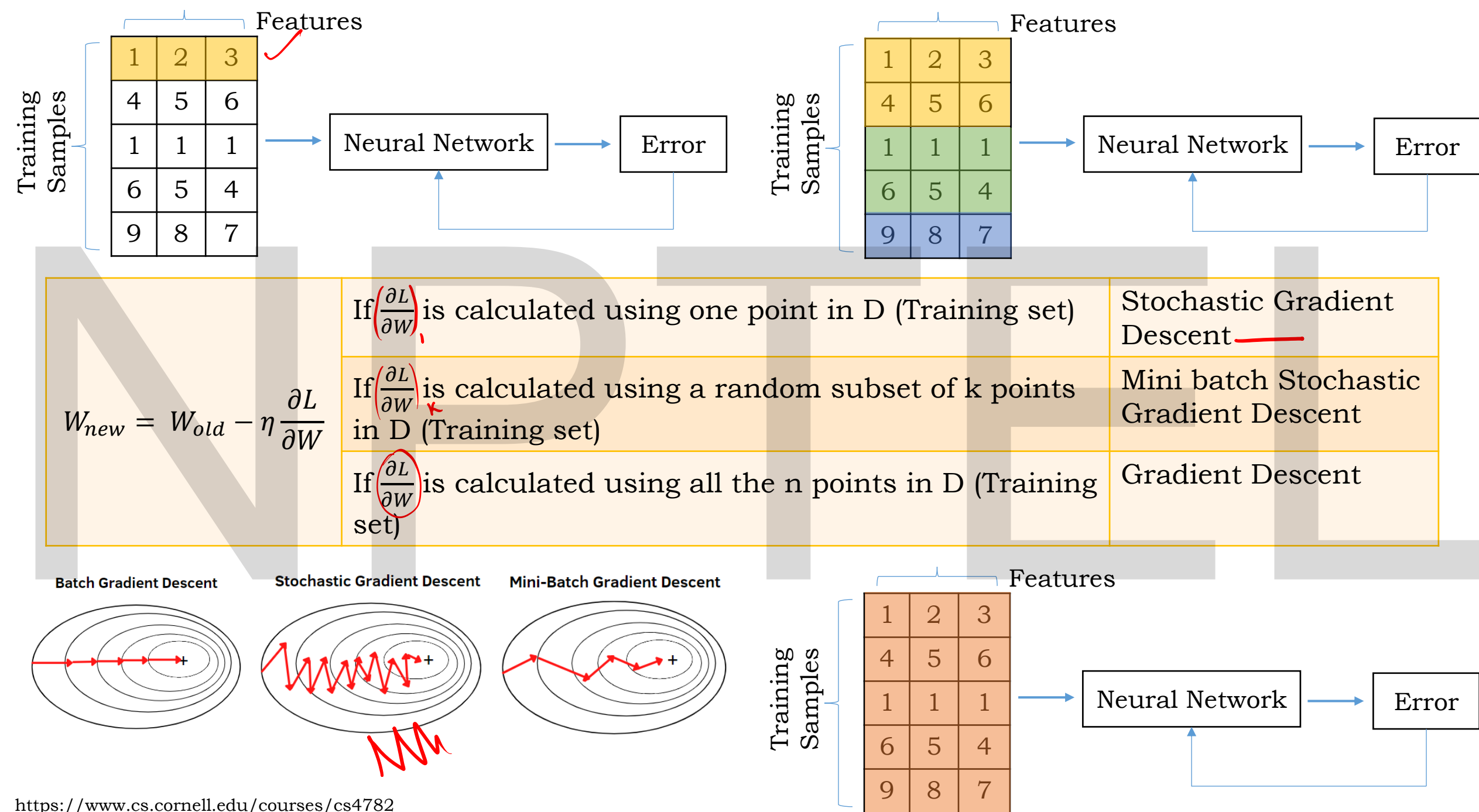
Once the loss is calculated, the next step is to find out **how each weight contributed to the loss**.

So, the gradient gives both:

- the **direction** of weight update,
- and the **amount of influence** that weight has on the loss.

Optimizers





SGD with Momentum(Polyak, 1964)

Compute an Exponentially Weighted Moving Average (EWMA) of the gradients as momentum and use that to update the weight instead.

SGD Update Rule:

$$W_{new} = W_{old} - \eta \left(\frac{\partial L}{\partial W} \right)_1 \longrightarrow$$

In **normal SGD**, the weight update uses only the **current gradient**.

$$W_{new} = W_{old} - \eta v_t$$

to get a *smoothed direction of movement* using both:
past gradients + current gradient

$$v_t = \beta v_{t-1} + (1 - \beta) g_t$$

where:

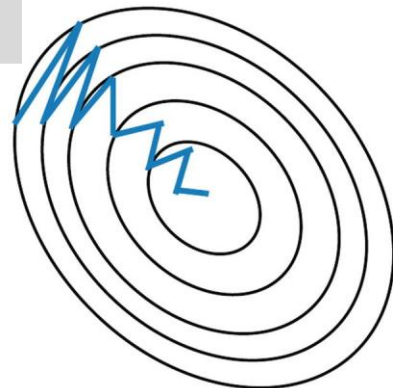
- v_t = momentum / velocity at step t
- β = momentum coefficient, usually close to 1 such as 0.9
- $g_t = \frac{\partial L}{\partial w_t}$ = current gradient

This is the **Exponentially Weighted Moving Average (EWMA)** of past gradients.

Exponentially weighted means recent gradients are given more importance, while older gradients are still remembered but with decreasing importance.



SGD without Momentum



SGD with Momentum

Small numerical intuition

Suppose:

$$\beta = 0.9$$

At step 1, let:

$$v_0 = 0$$

$$g_1 = 5$$

Then:

$$v_1 = 0.9(0) + 0.1(5)$$

$$v_1 = 0.5$$

At step 2, suppose:

$$g_2 = 6$$

$$v_2 = 0.9(0.5) + 0.1(6)$$

$$v_2 = 0.45 + 0.6$$

$$v_2 = 1.05$$

At step 3, suppose:

$$g_3 = 7$$

$$v_3 = 0.9(1.05) + 0.1(7)$$

$$v_3 = 0.945 + 0.7$$

$$v_3 = 1.645$$

So, if gradients keep pointing in the same direction, the velocity increases gradually. This means momentum helps the optimizer move faster in that consistent direction.

$$W_{new} = W_{old} - \eta v_t$$

$$v_t = \beta v_{t-1} + (1 - \beta)g_t$$

Now suppose suddenly one noisy gradient comes:

$$g_4 = -4$$

Then:

$$v_4 = 0.9(1.645) + 0.1(-4)$$

$$v_4 = 1.4805 - 0.4$$

$$v_4 = 1.0805$$

Even though the current gradient is negative, the velocity does not immediately become negative. It only reduces slightly.

This is the smoothing effect.

NIPTELL

Thank you